

Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

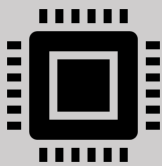
División de Ciencias de la ingeniería

Ingeniería

Organización de Lenguajes y Compiladores 2 Sección “A”

Ing. Mario Moises Ramirez Tobar

Segundo Semestre 2026



Estudiante:

Pablo Alejandro Maldonado de León

Carné: 202430233

Manual de usuario Práctica #1



Descripción:

En el siguiente manual se describe paso a paso cómo realizar la instalación de la aplicación, esta consiste en un validador y traductor de un lenguaje de programación en latín y permite la visualización de los procesos y diferentes fases que realiza un compilador para llegar a procesar un código de entrada en cierto lenguaje. También esta aplicación está realizada con una interfaz gráfica intuitiva para el usuario donde el puede acomodar las vistas como le sean útiles y en ningún momento se le obliga a tener cierta pestaña abierta. Espero sea de su agrado.

“Bienvenido usuario”

Como primer punto se requieren ciertos recursos básicos para poder ejecutar el programa. Debido a las tecnologías usadas como aplicación de escritorio, esta cuenta con procesos optimizados lo cual no conlleva una demanda bastante grande en requerimientos para ejecutar la aplicación.

Recomendaciones:

Debido a que la aplicación está diseñada para escritorio se debe tomar en cuenta que no está disponible para dispositivos móviles a no ser que el usuario desee correr allí. Los requerimientos básicos son los siguientes:

- Teclado y mouse.
- El sistema operativo queda a decisión del usuario.
- **Una pantalla lo suficientemente grande para poder visualizar con comodidad las diferentes representaciones gráficas que contiene la aplicación.**

Pasos iniciales:

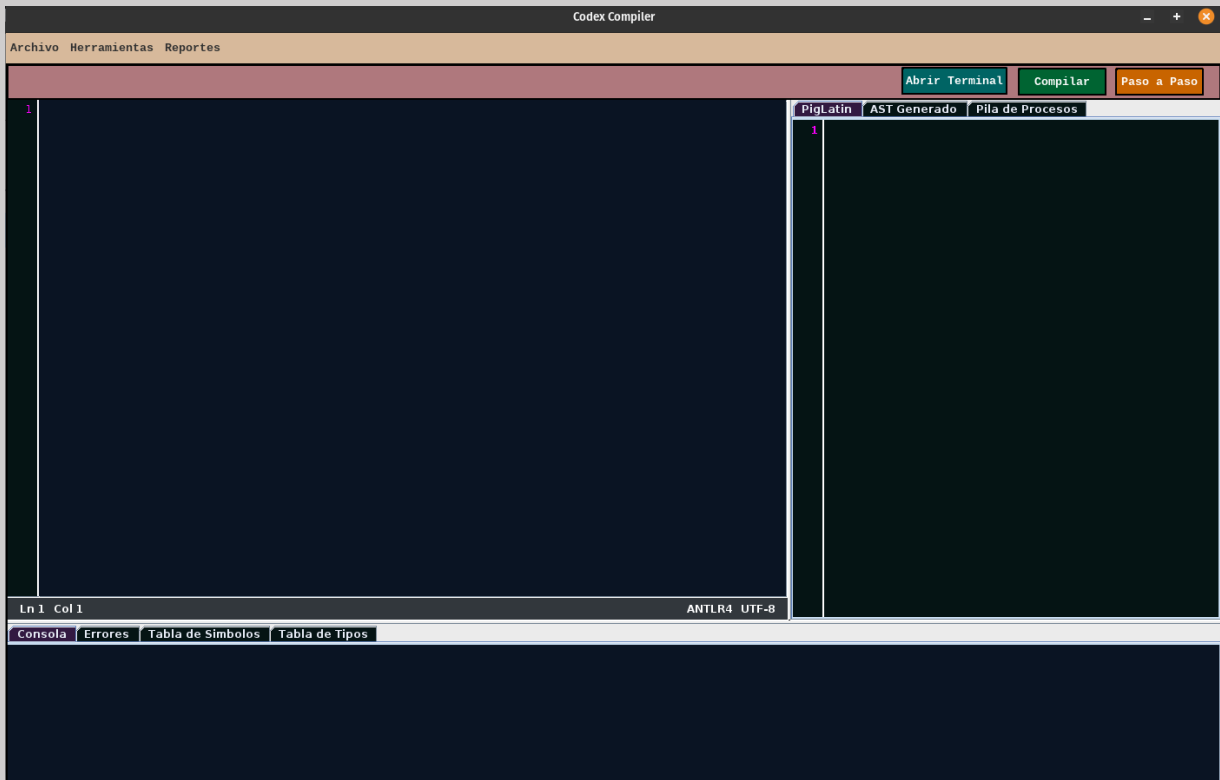
1. Encender el dispositivo
2. Descargar el archivo .zip o clonar el repositorio
3. Extraer el archivo comprimido o navegar hasta el directorio destino
4. En la raíz del programa o en la carpeta destino abierta.
5. Ejecutar el comando **mvn clean package**
6. Navegar hasta la carpeta que se va a crear llamada build y entrar a la carpeta **target**
7. Abrir la terminal y ejecutar el siguiente comando

java -jar practica-no1-compi2-1.0.0.jar

Si en dado caso la aplicación le pide permisos para poder acceder a sus archivos **debe darle aceptar ya que esta tiene la opción de poder cargar archivos siempre y cuando tengan la extensión .lat y exportar archivos .pig**

MENÚ PRINCIPAL

Apenas ejecutar el comando en la terminal del dispositivo saltará la interfaz de inicio lo que permite generar una primera vista de cómo está hecha la interfaz. El tema está basado completamente en un tema oscuro para evitar ciertos problemas de visualización con un tema claro:



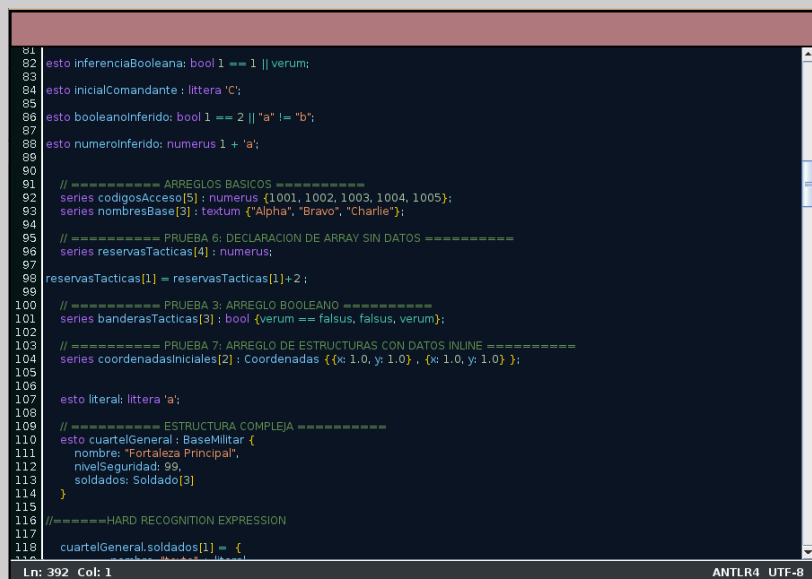
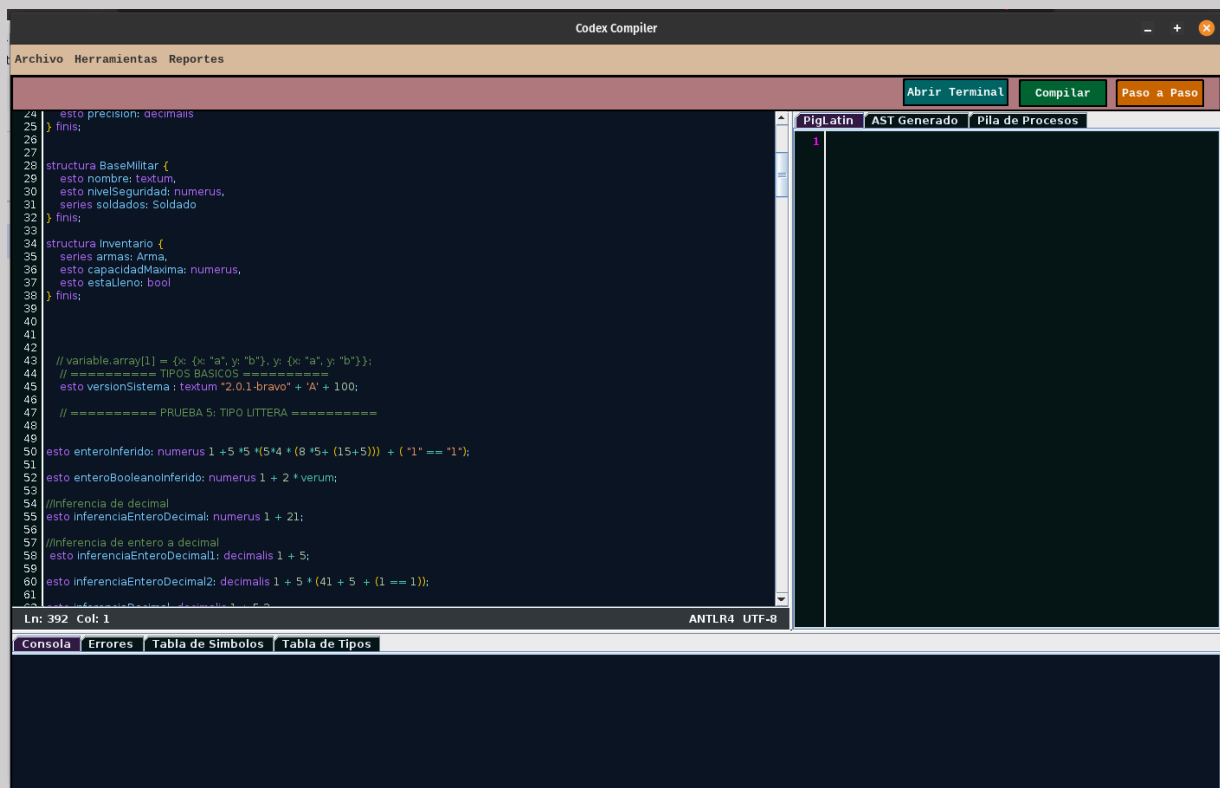
Nota:

La interfaz de usuario cuenta con una tipografía específica llamada “Liberation mono” que da esa estética retro pero a la vez elegante de un lenguaje de programación. También no está demás tomar en cuenta que:

“Todas las vistas son redimensionables y ajustables a gusto del usuario”

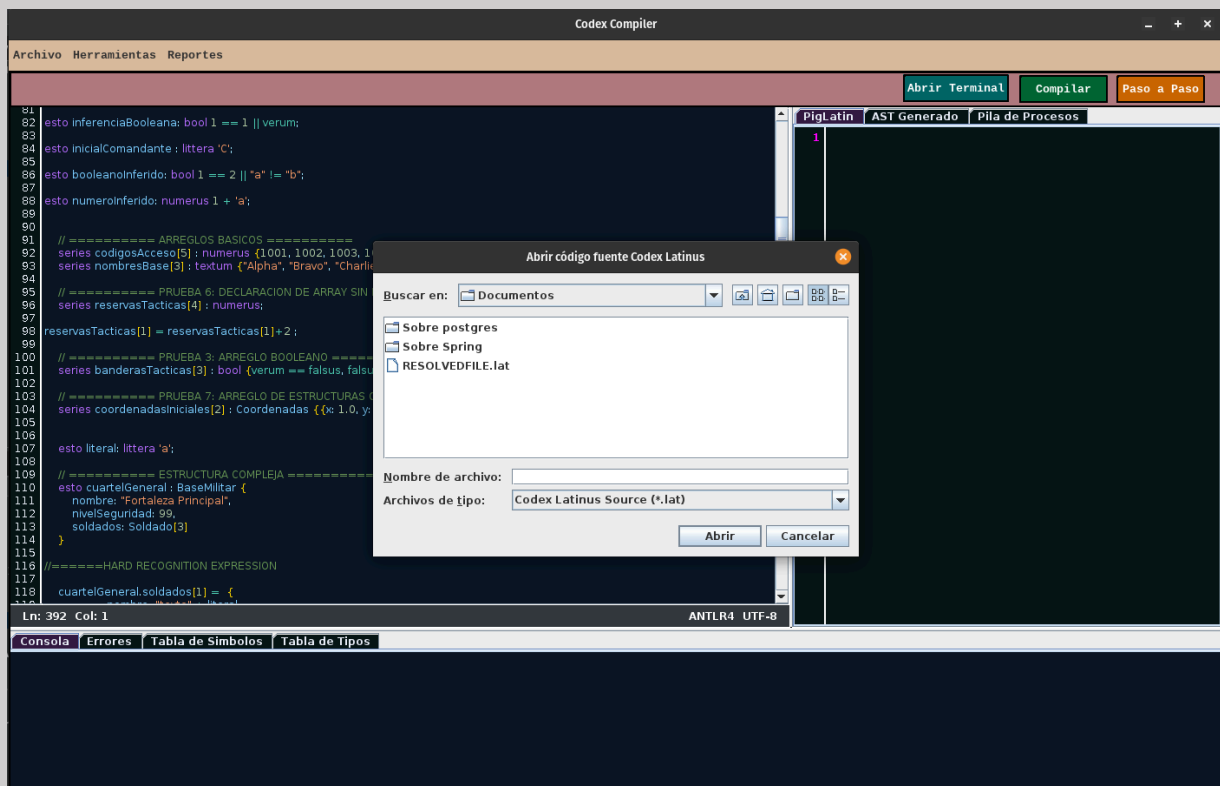
COLOREADO DINAMICO DEL CÓDIGO

Algo que se podrá notar apenas empezar a escribir el código en la aplicación. Es que cuenta con coloreado dinámico para poder tener una mejor vista de las palabras reservadas y valores a la hora de programar, brindando una interfaz amigable.



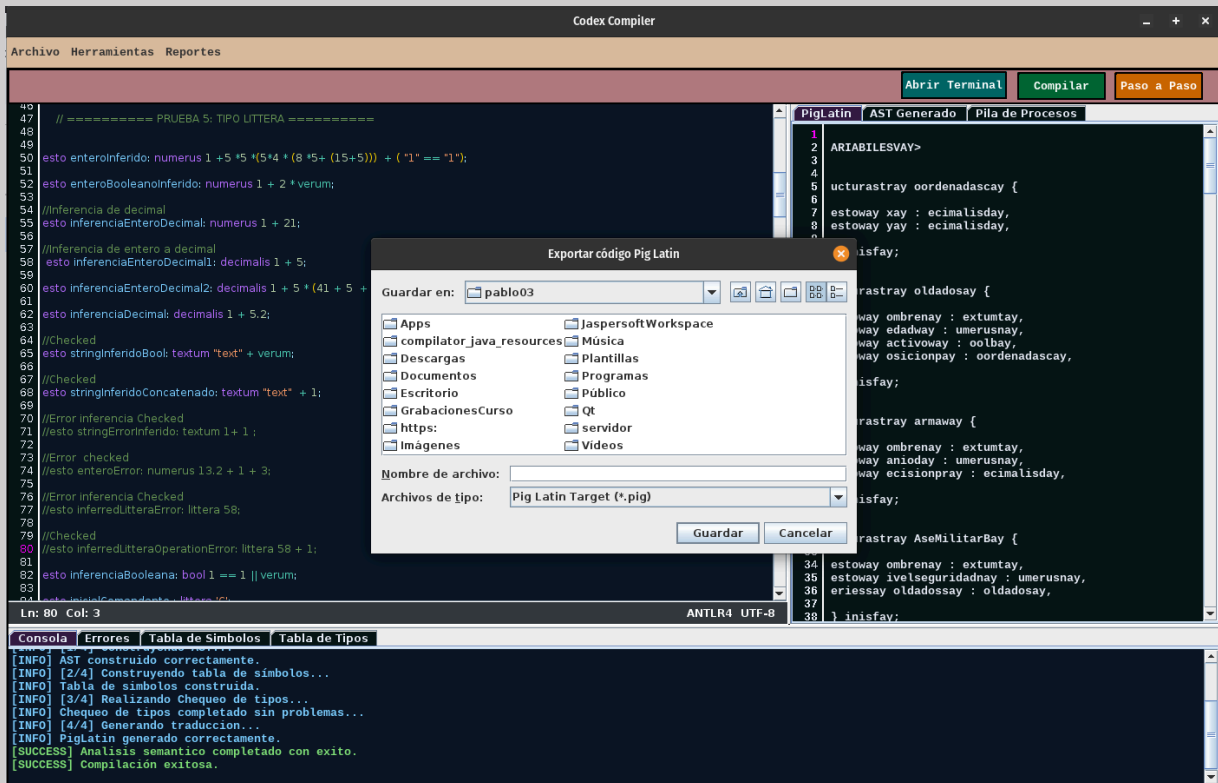
IMPORTACIÓN DE ARCHIVOS

La aplicación permite poder importar archivos con extensión **.lat**, por lo que se deberá desplazar a la parte superior izquierda de la pantalla y se deberá elegir la opción de “Abrir un archivo”.



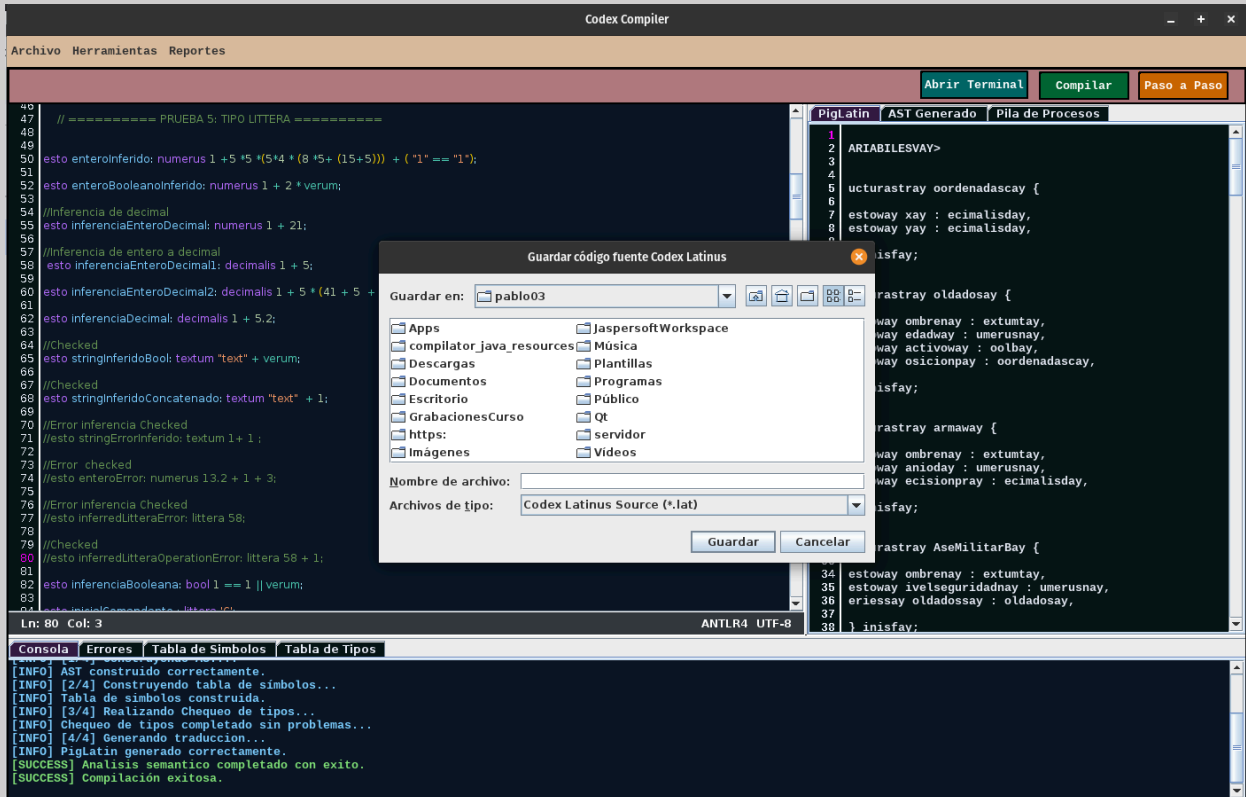
EXPORTACIÓN DE ARCHIVOS

La aplicación permite poder exportar archivos con extensión **.pig**, esto únicamente se aplica para cuando se realiza la compilación. Por lo que se deberá desplazar a la parte superior izquierda de la pantalla y se deberá elegir la opción de “Descargar código”.



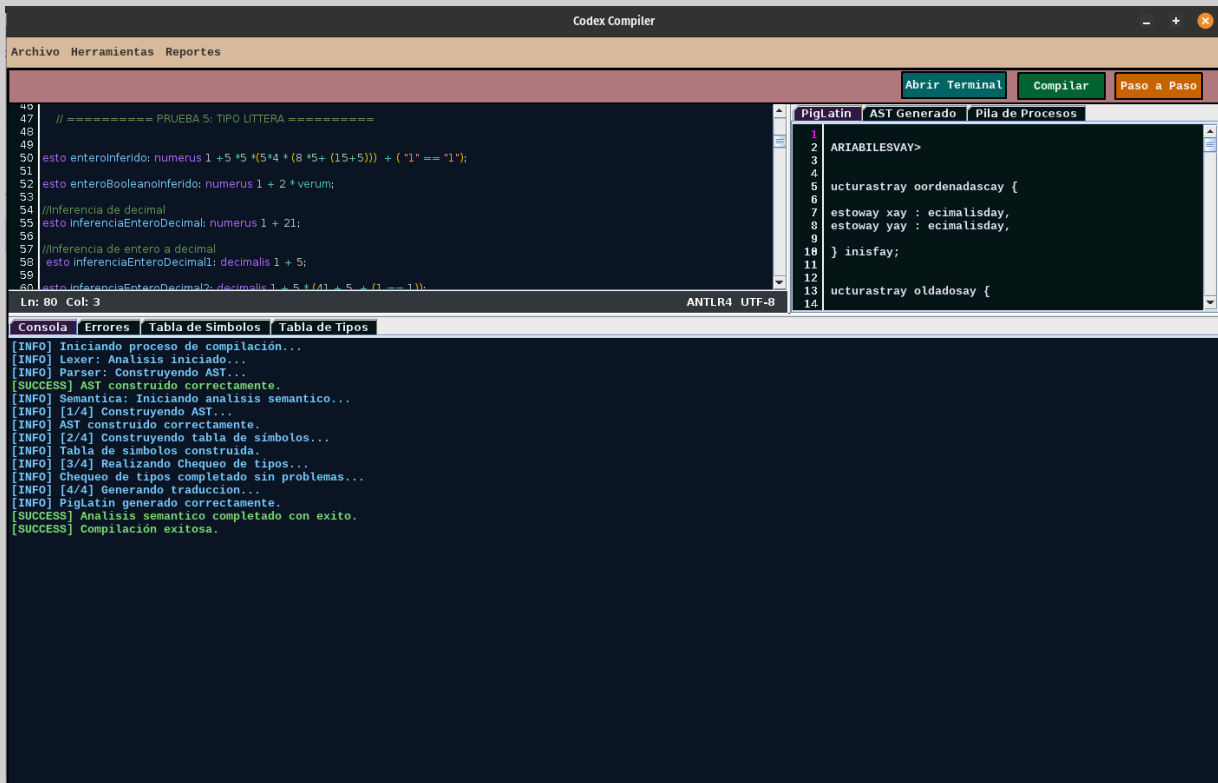
GUARDADO DE ARCHIVOS

La aplicación permite poder guardar el código que se ha programado en la aplicación con extensión **.lat**, esto con la finalidad de que el usuario pueda guardar su código para seguir programando después. Por lo que se deberá desplazar a la parte superior izquierda de la pantalla y se deberá elegir la opción de “Guardar código”.



CONSOLA DE INSTRUCCIONES

La aplicación cuenta con la vista de una consola que comunicará en todo momento lo que esté sucediendo en la aplicación o si se encuentra con algún error en alguna fase de compilación. O en especial si se tiene cierto problema con algo de la interfaz.



The screenshot displays the Codex Compiler application window. The main editor shows a source code file with line numbers 47 to 60. The code includes comments and a complex arithmetic expression. To the right, the 'AST Generado' tab shows the generated PigLatin code, which includes variable declarations and function definitions. At the bottom, the 'Consola' tab displays a log of compilation steps, from starting the process to successful compilation.

```
47 // ===== PRUEBA 5: TIPO LITTERA =====
48
49
50 esto enteroinferido: numerus 1 + 5 * 5 * (5 * 4 * (8 * 5 + (15 + 5))) + ( '1' == "1");
51
52 esto enteroBooleaninferido: numerus 1 + 2 * verum;
53
54 //Inferencia de decimal
55 esto inferenciaEnteroDecimal: numerus 1 + 21;
56
57 //Inferencia de entero a decimal
58 esto inferenciaEnteroDecimal1: decimalis 1 + 5;
59
60 esto inferenciaEnteroDecimal2: decimalis 1 + 5 + (41 + 5 + (1 == 1));
```

Ln: 80 Col: 3 ANTLR4 UTF-8

PigLatin | **AST Generado** | **Pila de Procesos**

```
1
2 ARIABLESVAY>
3
4
5 ucturastray oordenadascay {
6
7 estoway xay : ecimalisday,
8 estoway yay : ecimalisday,
9
10 } inisfay;
11
12
13 ucturastray oldadosay {
14
```

Consola | **Errores** | **Tabla de Símbolos** | **Tabla de Tipos**

```
[INFO] Iniciando proceso de compilación...
[INFO] Lexer: Analisis iniciado...
[INFO] Parser: Construyendo AST...
[SUCCESS] AST construido correctamente.
[INFO] Semantica: Iniciando analisis semantico...
[INFO] [1/4] Construyendo AST...
[INFO] AST construido correctamente.
[INFO] [2/4] Construyendo tabla de simbolos...
[INFO] Tabla de simbolos construida.
[INFO] [3/4] Realizando Chequeo de tipos...
[INFO] Chequeo de tipos completado sin problemas...
[INFO] [4/4] Generando traduccion...
[INFO] PigLatin generado correctamente.
[SUCCESS] Analisis semantico completado con exito.
[SUCCESS] Compilación exitosa.
```


TABLA DE ERRORES

La aplicación cuenta con la vista de una tabla que mostrará en todo momento los errores que se detecten en las diferentes fases y analizadores con los que cuenta el compilador. Se puede acceder desde las instrucciones de la parte superior izquierda o directamente haciendo click sobre las pestañas del panel inferior.

The screenshot shows the Codex Compiler application window. The main editor displays a code snippet with line numbers 38 to 51. The code includes comments in Spanish and a line with a syntax error: `esto enteroinferido: numerus 1 + 5 * 5 * (5 * 4 * (8 * 5 + (15 + 5))) + ( '1' == "1" );`. The error table at the bottom shows three errors related to this line.

Lexema	Línea	Columna	Tipo	Descripción
@	51	1	Lexico	Caracter no definido en el lenguaje
+	58	43	Sintactico	Símbolo no esperado '+'. Se esperaba {'verum', 'falsus', '{', ...
@	81	0	Sintactico	Se esperaba {'MUNERA', 'MAIOR'} pero se encontró '@'

Consola Errores Tabla de Símbolos Tabla de Tipos

Compilacion fallida

TABLA DE SÍMBOLOS

La aplicación cuenta con la vista de una tabla que mostrará en todo momento los símbolos o identificadores que se detectan en la fase de análisis semántico, sobre todo se muestra como una representación gráfica de cómo se va armando o actúa internamente el compilador para poder evaluar los identificadores y registrar funciones. Se puede acceder desde las instrucciones de la parte superior izquierda o directamente haciendo click sobre las pestañas del panel inferior.

The screenshot shows the Codex Compiler interface. The main editor displays a code snippet with line numbers 38 to 51. The right panel shows the AST (AST Generado) with a tree structure. The bottom panel shows the Symbol Table (Tabla de Símbolos) with columns: ID, Tipo de Símbolo, Tipo de Dato, Ámbito, Línea, and Columna.

ID	Tipo de Símbolo	Tipo de Dato	Ámbito	Línea	Columna
inferenciaEnteroDecimal1	VARIABLE	decimalis	Global	58	1
inferenciaEnteroDecimal2	VARIABLE	decimalis	Global	60	0
inferenciaDecimal	VARIABLE	decimalis	Global	62	0
stringInferidoBool	VARIABLE	textum	Global	65	0
stringInferidoConcatenado	VARIABLE	textum	Global	68	0
inferenciaBooleana	VARIABLE	bool	Global	82	0
inicialComandante	VARIABLE	littera	Global	84	0
booleanoInferido	VARIABLE	bool	Global	86	0
numeroInferido	VARIABLE	numerus	Global	88	0
codigosAcceso	ARRAY	numerus	Global	92	4
nombresBase	ARRAY	textum	Global	93	4
reservasTacticas	ARRAY	numerus	Global	96	4
banderasTacticas	ARRAY	bool	Global	101	4
coordenadasIniciales	ARRAY	Coordenadas	Global	104	4
literal	VARIABLE	littera	Global	107	4
cuartelGeneral	VARIABLE	BaseMilitar	Global	110	4

TABLA DE TIPOS

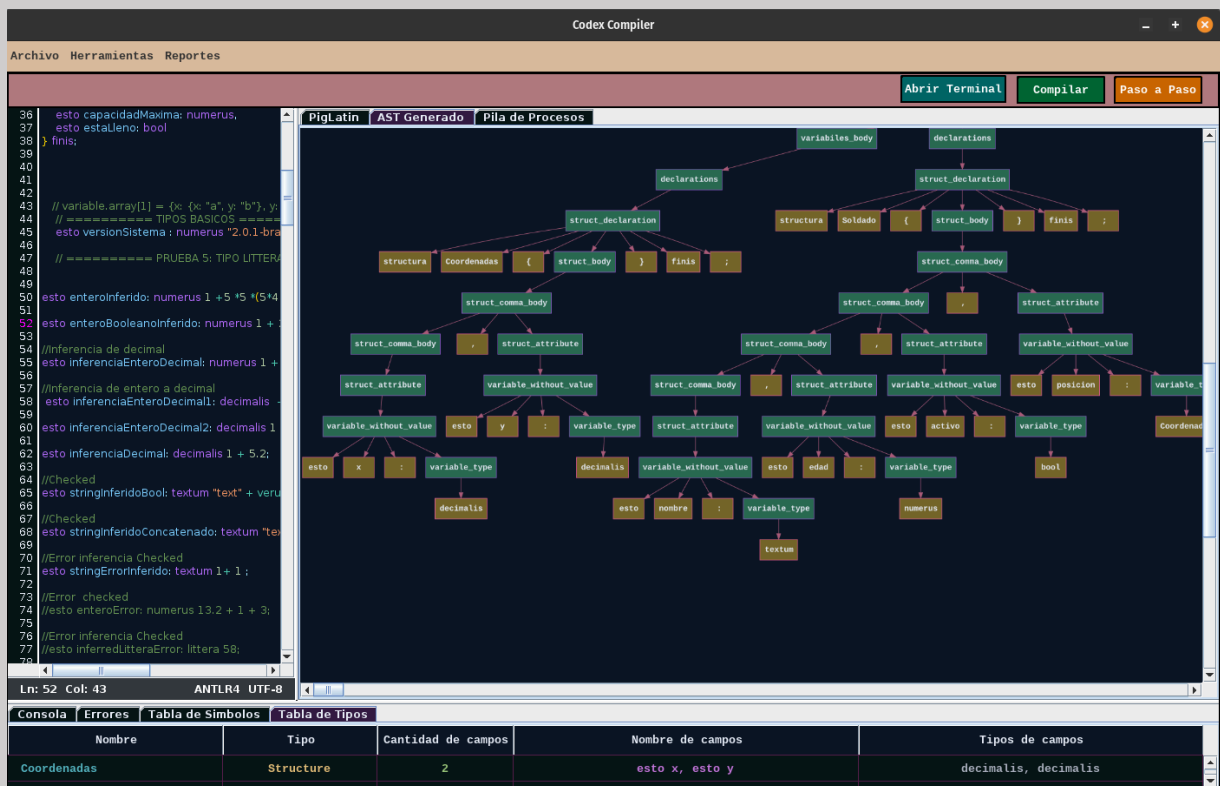
La aplicación cuenta con la vista de una tabla que mostrará en todo momento los tipos personalizados que se detectan en la fase de análisis semántico, sobre todo se muestra como una representación gráfica de cómo se va armando o actúa internamente el compilador para poder evaluar los diferentes tipos personalizados con los que se va a contar al momento de declarar las variables. Se puede acceder desde las instrucciones de la parte superior izquierda o directamente haciendo click sobre las pestañas del panel inferior.

The screenshot shows the Codex Compiler interface. The main editor displays code with comments in Spanish. The 'Tabla de Tipos' (Type Table) panel is active, showing a table with the following data:

Nombre	Tipo	Cantidad de campos	Nombre de campos	Tipos de campos
Coordenadas	Structure	2	esto x, esto y	decimalis, decimalis
Soldado	Structure	4	esto nombre, esto edad, esto activo, esto pos...	textum, numerus, bool, Coordenadas
Arma	Structure	3	esto nombre, esto dano, esto precision	textum, numerus, decimalis
BaseMilitar	Structure	3	esto nombre, esto nivelSeguridad, series sold...	textum, numerus, Soldado
Inventario	Structure	3	series armas, esto capacidadMaxima, esto esta...	Arma, numerus, bool

REPRESENTACIÓN DEL ÁRBOL DE DERIVACIÓN

La aplicación cuenta con la vista del árbol de derivación que genera el analizador sintáctico al momento de procesar los tokens y todos los no terminales definidos en su respectiva programación. Esto le permite al usuario darse una idea de cómo ocurre el proceso de programación de un compilador para el reconocimiento de secuencia de tokens. Se puede acceder desde las instrucciones de la parte superior izquierda o directamente haciendo click sobre las pestañas del panel inferior.



PILA DE PROCESOS

Este es el proceso más bonito con el que cuenta la aplicación ya que simula cómo se comporta una pila de análisis sintáctico, y esta es una característica importante de la herramienta utilizada ya que como tal para llegar a una representación o análisis correcto sintácticamente pasa por simplemente una pila de análisis donde se desafilan y apilan procesos hasta llegar a la cadena de aceptación.

The screenshot shows the Codex Compiler interface. The main window is titled 'Codex Compiler' and has a menu bar with 'Archivo', 'Herramientas', and 'Reportes'. Below the menu bar are buttons for 'Abrir Terminal', 'Compilar', and 'Paso a Paso'. The main area is divided into two panes. The left pane shows the source code with line numbers 36 to 76. The right pane is titled 'Pila de Procesos' and contains a 'Visualizador de Pila' (Stack Visualizer) with 6 slots. Each slot has a button to shift a specific element onto the stack: 'shift VARIABLES', 'shift >', 'shift estructura', 'shift Coordenadas', and 'shift {'. Below the stack visualizer are buttons for 'Reiniciar', 'Atrás', 'Siguiente', and 'Terminar'. The bottom pane is titled 'Salida de Procesos' and shows the sequence of steps: [Paso 1] -> Pila Vacía, [Paso 2] -> shift VARIABLES, [Paso 3] -> shift >, [Paso 4] -> shift estructura, [Paso 5] -> shift Coordenadas, [Paso 6] -> shift {.

Nombre	Tipo	Cantidad de campos	Nombre de campos	Tipos de campos
Coordenadas	Structure	2	esto x, esto y	decimalis, decimalis

Las opciones que ofrece son:

- **Reiniciar:** Devuelve la pila al estado inicial y limpia todos los logs realizados.
- **Atrás:** Permite regresar los pasos en los que se va ejecutando la pila de procesos.
- **Siguiente:** Permite ir avanzando a discreción del usuario sobre los diferentes procesos que realiza el analizador sintáctico.
- **Terminar:** Viaja hacia el final de la pila dejando a su paso todos los estados y procesos de ilustración del análisis sintáctico

TRADUCCION DEL CODIGO

Esta es la última parte y sección que ofrece la aplicación ya que toda la aplicación y compilador está basada en una traducción de un lenguaje en latín a un lenguaje de cerdos. Lo cual permite que se pueda evidenciar la traducción dirigida por la sintaxis, como una pseudo representación de código intermedio gracias al recorrido interno del AST.

Codex Compiler

Archivo Herramientas Reportes

Abrir Terminal Compilar Paso a Paso

PigLatin **AST Generado** **Pila de Procesos**

```
36 esto capacidadMaxima: numerus,
37 esto estaLleno: bool
38 } finis;
39
40
41
42
43 // variable.array[1] = {x: {x: "a", y: "b"}, y: {x: "a",
44 // ===== TIPOS BASICOS =====
45 esto versionSistema: numerus "2.0.1-bravo" + "A";
46 // ===== PRUEBA 5: TIPO LITTERA =====
47
48
49
50 esto enteroInferido: numerus 1 + 5 * 5 * (5 + 4 * (8 * 5
51
52 esto enteroBooleanoInferido: numerus 1 + 2 * verum;
53
54 //Inferencia de decimal
55 esto inferenciaEnteroDecimal: numerus 1 + 21;
56
57 //Inferencia de entero a decimal
58 esto inferenciaEnteroDecimal: decimalis + 5;
59
60 esto inferenciaEnteroDecimal2: decimalis 1 + 5 * (
61
62 esto inferenciaDecimal: decimalis 1 + 5.2;
63
64 //Checked
65 esto stringInferidoBool: textum "text" + verum;
66
67 //Checked
68 esto stringInferidoConcatenado: textum "text" + 1;
69
70 //Error inferencia Checked
71 esto stringErrorInferido: textum 1 + 1;
72
73 //Error checked
74 //esto enteroError: numerus 13.2 + 1 + 3;
75
76 //Error inferencia Checked
77 //esto inferidoLitteraError: littera 58;
78
```

Ln: 52 Col: 43 ANTLR4 UTF-8

Consola **Errores** **Tabla de Símbolos** **Tabla de Tipos**

Nombre	Tipo	Cantidad de campos	Nombre de campos	Tipos de campos
Coordenadas	Structure	2	esto x, esto y	decimalis, decimalis